Existing compilers miss useful optimizations

A useful rewrite may fail to match or run at the wrong stage. Other opportunities need a new transformation.

Rule / matching quality

An available rewrite may fail to match the IR form produced by the current pipeline.

Pass placement / ordering

A producing pass may expose ranges, precision or other facts after a consuming pass has run.

New pass opportunities

A useful transformation or coordinated sequence may need a new rule, analysis or pass.

Find useful IR alternatives and pass sequences. Compare the complete candidate route against the recorded baseline under the selected checks and metric.

Rule quality, pipeline placement and missing transformations are investigation hypotheses, not confirmed Ascend diagnoses.

[REWORK] How optimization opportunities reach engineers today

Current mechanism	Trigger and engineering work
Nightly performance regression testing	Detect deterioration against a baseline before upstream integration. Investigate or revert the offending change.
Performance issue tickets	Investigate slow cases discovered in already merged code.
Kernel-specific tasks	Inspect generated code and profiling data to make a specified kernel faster.
Optimization epics	Proactively explore an optimization class, usually in response to an observed performance gap.

The gap: routine discovery of optimization opportunities before a performance problem is noticed. (To my knowledge)

Routine discovery needs a repeatable investigation process



Representative repeated investigation activities

Work	Material an engineer needs
Understand the region	Definitions, uses, effects, aliasing, shape/layout and numeric constraints.
Test an alternative	Compiler entry point, enclosing module, target settings, harness and inputs.
Assess the result	Comparable output artifacts, selected metric, conditions and reproduction steps.

Build a framework for reproducible optimization discovery

Develop a framework that searches IR and pass-pipeline alternatives and returns reproducible optimization experiments for AscendNPU-IR.



Offload variant construction, experiment execution and evidence collection. Measure whether useful findings justify preparation and review effort.

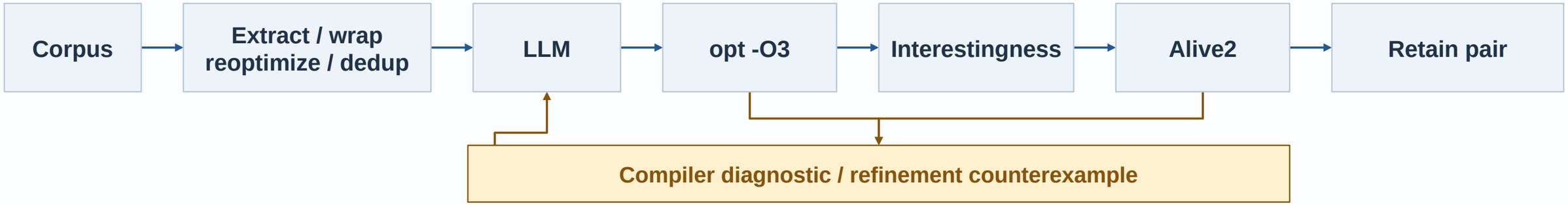
Research directions and concrete starting points

Study	Concrete starting points	What to compare	Evidence to retain
Pass / rewrite search	MLIR textual pass pipelines. IR variants + scoped pass edits.	Rewrite only, ordering only and combined candidates.	Full pass sequences, checkpoints and paired results.
Improvement metrics	MLIR op statistics. LLVM TTI / llvm-mca. Ascend msProf.	Cheap indicators vs downstream code or execution.	Coverage, downstream survival and collection cost.
Compilation / checks	MLIR verifier / Alive2. MLIR-TV / xdsl-smt.	Checkpoint compatibility and cross-pass semantic coverage.	Route matrix, assumptions and reproducible probes.
Agent context	Codex / Claude Code / OpenCode. SQL agents / PaperQA.	Direct dialect context vs retrieval or added analyses.	Useful findings, invalid edits and context cost.
Agent tools	File edits and diagnostics. Operation-aware editor.	IR / pipeline edit controls and checking tools.	Valid proposals, reachable changes and total cost.
Region discovery	Use-def / effect scans. Cost or profile estimates.	Supplied range vs prompted, static or cost-guided selection.	Coverage, useful yield and selection cost.
Feedback / policy	LPO errors / counterexamples. PRAGMA profiles and history.	Feedback detail, continuation and budget allocation.	Attempt traces, useful yield and expensive checks.

Candidate tools and mechanisms to investigate. Ascend integration and useful semantic / metric coverage remain unestablished.

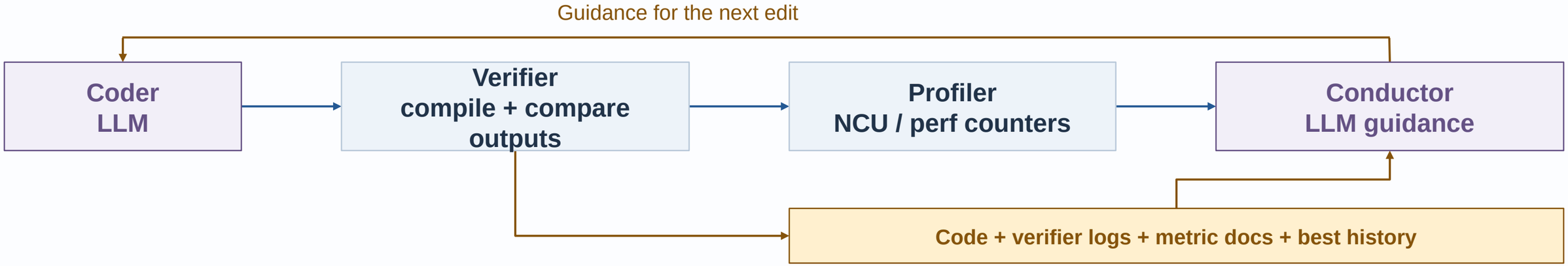
Two paper workflows inform the experiment

LPO: Discovering Missed Peephole Optimizations with Large Language Models



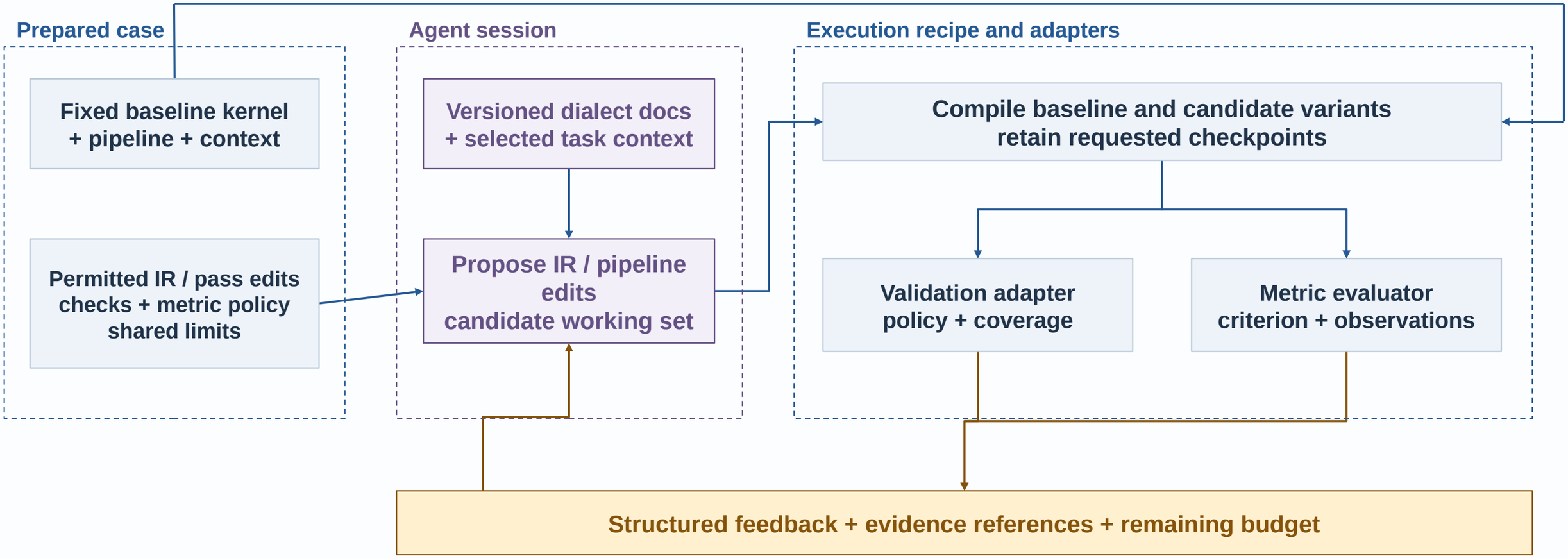
Screen uses instruction count or llvm-mca estimates; syntactic alternatives may also be retained.

PRAGMA: A Profiling-Reasoned Multi-Agent Framework for Automatic Kernel Optimization



Editable reconstructions of LPO Fig. 2 and PRAGMA Figs. 1–2. LLVM and CPU/GPU tool assumptions belong to the papers.

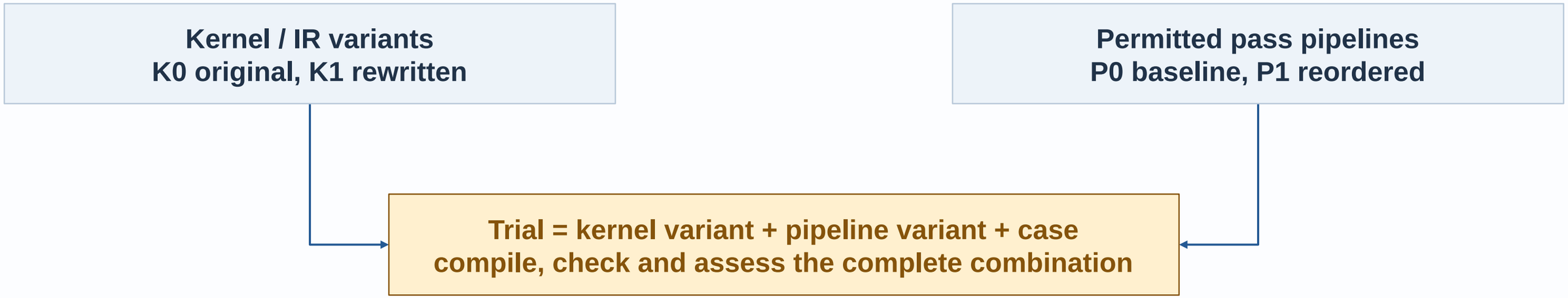
Proposed Ascend investigation architecture



Persist kernels, pipelines, trials, results, agent reports, pending work and budgets across every stage.

Finding eligibility = required checks pass + selected criterion met + complete runnable bundle.

Search combines kernel variants and pass pipelines



Experiment	Question / comparison
K0 + P0	Fixed baseline under recorded compiler, target and workload settings.
K0 + P1	Does pass placement / ordering expose an existing optimization?
K1 + P0, then K1 + P1	Does the IR change help alone or together with the pipeline change?

Example: track ranges / cast precision, then run the consuming simplification. Check pass invariants and downstream output.

Candidate transformations and applicability conditions

Family	Opportunity to investigate	Conditions / checkpoint information
Value ranges / precision	Track ranges and precision through casts to expose simplifications.	Signedness, rounding / overflow, cast semantics and observed results.
Predicates / selects	Remove a NaN guard before ordered equality.	FP predicate semantics and other uses of the sanitized value.
Memory / UB across passes	Combine accesses only with covered cross-pass definedness assumptions.	Bounds, alignment, address space, effects and poison/undef or dialect semantics after lowering.
Vectorization / lowering	Expose a target instruction or change vectorization choices.	Vector width, legal operations and contraction permissions.
Reduction organization	Change reduction structure or target mapping.	Shape, neutral element, NaN handling and numeric order.
Layout conversions	Eliminate inverse changes or retain a layout across consumers.	Physical layout, aliasing and downstream requirements.
Pass placement / ordering	Move or repeat an existing pass after enabling facts / forms appear.	Pass invariants, required analyses / invalidation and remaining pipeline.

Memory / UB work is a later scope candidate until semantic coverage is demonstrated. Other families also require complete-route checks.

MLIR case — redundant NaN handling

Shared operands %x : f32 %zero = 0.0 : f32 %one = 1.0 : f32

Original

```
%ordinary = arith.cmpf ord, %x, %x : f32
%safe = arith.select %ordinary, %x, %zero : f32
%equal = arith.cmpf oeq, %safe, %one : f32
```

Candidate

```
%equal = arith.cmpf oeq, %x, %one : f32
```

Input class	Original result	Candidate result
NaN	select zero; 0.0 == 1.0 → false	ordered equality → false
Ordinary value	x == 1.0	x == 1.0

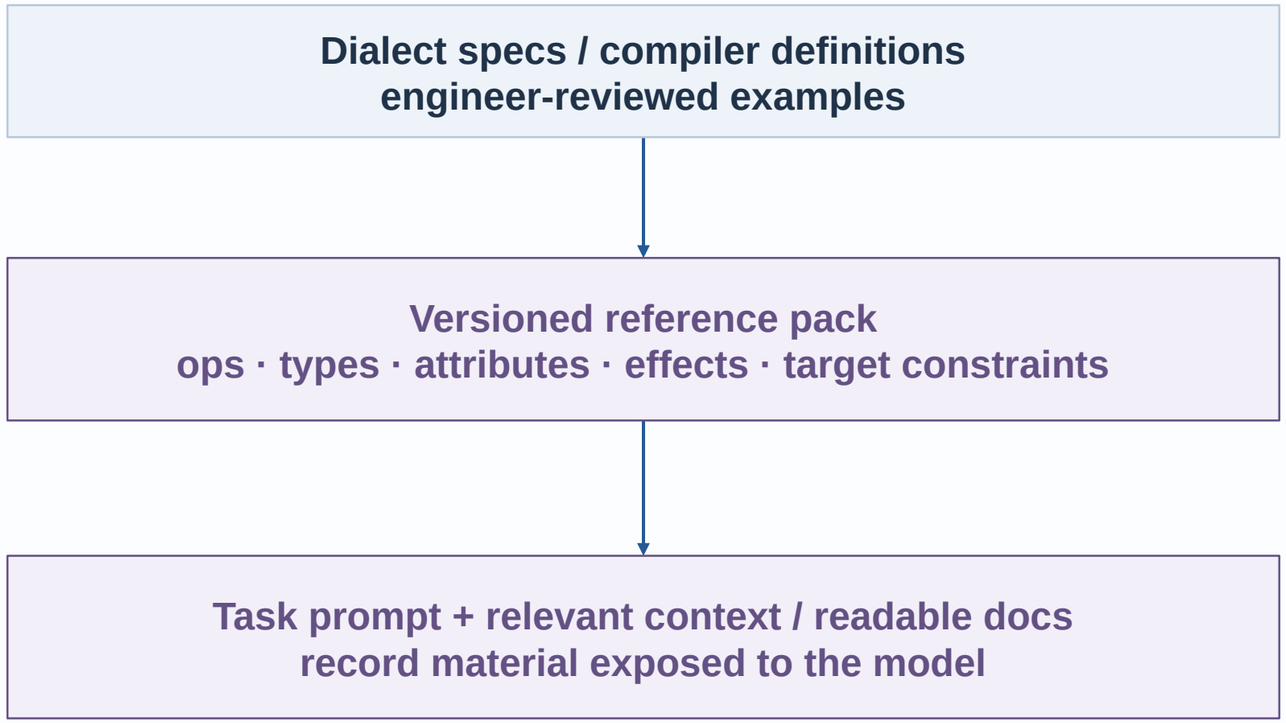
LPO Figure 4(c,f), adapted to MLIR. Inspect post-continuation output and the selected checks before retaining a finding.

Request resolution and dialect knowledge delivery

Illustrative request shape

```
input: module.mlir
mode: specified_range
region: operation_range
checkpoint: input_profile
pipelines: allowed_variants
checking: validation_profile
objective: evaluator_profile
limits: request_budget
```

Defaults → request → per-input overrides
Shared request limits stay upper bounds.



Prompt/context study: inspect public harnesses, then compare direct dialect context, retrieval and added analyses.

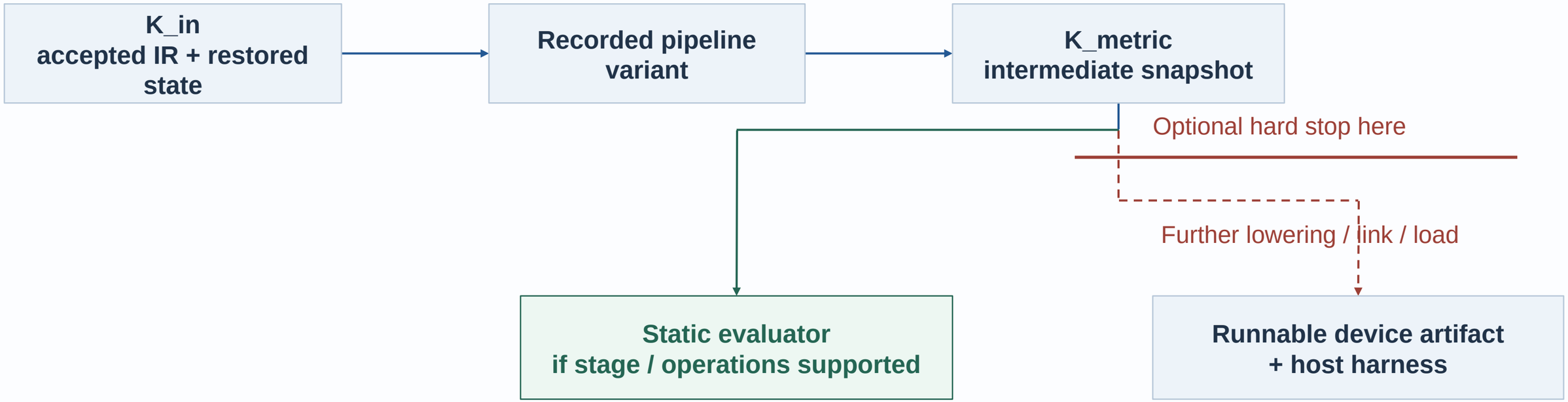
Agent context research: public harness starting points

Reference	Public mechanism to inspect	Possible Ascend experiment
Codex	Published starter prompt, repository instructions and tool use.	How to combine task instructions, dialect facts and compiler tools.
Claude Code	Custom subagent system prompts and scoped tool access.	Explicit IR-review role with a controlled tool set.
OpenCode	Agent definitions with custom prompts and tool permissions.	Separate reusable domain instructions from per-case context.
LangChain SQL agent	Schema retrieval and correction from database errors.	Retrieve operation semantics on demand and repair from diagnostics.
PaperQA	Evidence extraction and answer-synthesis prompt templates.	Summarize relevant dialect documentation while retaining sources.

Baseline: supplied IR + versioned dialect references. Compare direct context, retrieved documentation and targeted analyses under the same model and search budget.

Compilation checkpoints determine route compatibility

Illustrative continuation for each recorded baseline / candidate route



Runtime validation needs a runnable artifact. A hard stop before its required lowering blocks this policy. The metric checkpoint can remain earlier if continuation is allowed.

Route study output: checkpoint catalog, validation coverage and a demonstrated / incompatible / unknown configuration matrix.

Validation research: LLVM and MLIR starting points

Starting point	Input / available evidence	What the study must establish
MLIR verifier / project driver	Registered dialect IR and operation invariants.	Parsing / verification / compilation success is structural evidence.
Alive2 (LLVM)	LLVM IR original/candidate refinement check.	A usable LLVM route and supported semantics. Custom lowering needs separate justification.
MLIR-TV	SMT-based translation validation for supported MLIR.	Supported operations, shapes, memory and FP semantics for selected cases.
xdsl-smt / xdsl-tv	Refinement checks for subsets of arith, comb and func.	Representation mapping and the cost of missing operation semantics.
Execution comparison	Project harness, recorded inputs and observable outputs.	Runnable route, NPU requirements, tolerances and test coverage.

Retain a finding when the selected checks pass, the metric criterion is met and the replay bundle is complete. Record failed, unsupported and inconclusive checks separately.

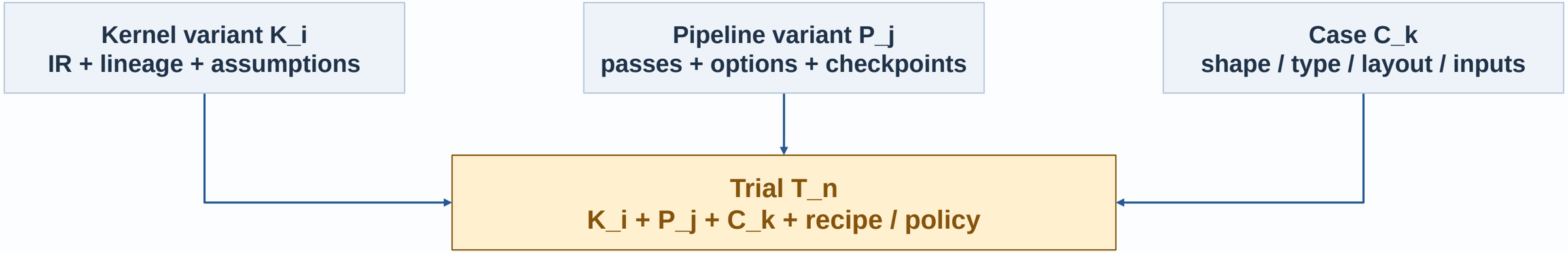
Metric research: concrete indicators and tools

Candidate metric	Starting point / resources	Interpretation and open checks
Operation counts	MLIR print-op-stats at a fixed checkpoint.	Cheap structural screen. Later lowering may erase or reverse the reduction.
Logical memory work	Shape/layout-based access and byte estimates.	Proposed indicator. Reuse, transfers and physical layout affect real traffic.
Target cost estimates	LLVM TTI for supported IR targets. llvm-mca for supported CPU assembly.	Study target / operation coverage. An equivalent Ascend model is not established.
Kernel time / counters	Ascend msProf / msprof op with a runnable operator.	Recorded durations / counter data. Check local tooling, warm-up, repeats and collection cost.



Metric improvement and preserved behavior require separate evidence. Static indicators alone do not establish runtime speedup.

A common experiment format joins variants and evidence



Record / proposed files	Contents linked through stable IDs
experiment.json + variants/	Schema version, baseline, kernel / pipeline / case variants and artifact hashes.
trials.jsonl + executions/	Trial combinations / recipes. Per-environment attempts, job state, dependencies and budgets.
evidence/	Check outcomes / coverage, metric samples / units, diagnostics and resource use.
agent/ + reports/	Supplied / retrieved context, exposed messages / tool results and agent-produced reports.

Portable records use bundle-relative artifact paths and declared external bindings. Exact schemas and migrations remain to design.

Feedback, stopping rules and crash recovery

Observed outcome	Feedback / next action
Compilation rejection or behavior mismatch	Send diagnostics or failing input with original/candidate identities. Agent may repair.
No qualifying improvement	Send kernel / pipeline / trial IDs, metric values and evidence references.
Unsupported check / unavailable hardware	Expose coverage or infrastructure cause; retain an unresolved operation.

Durable operation protocol

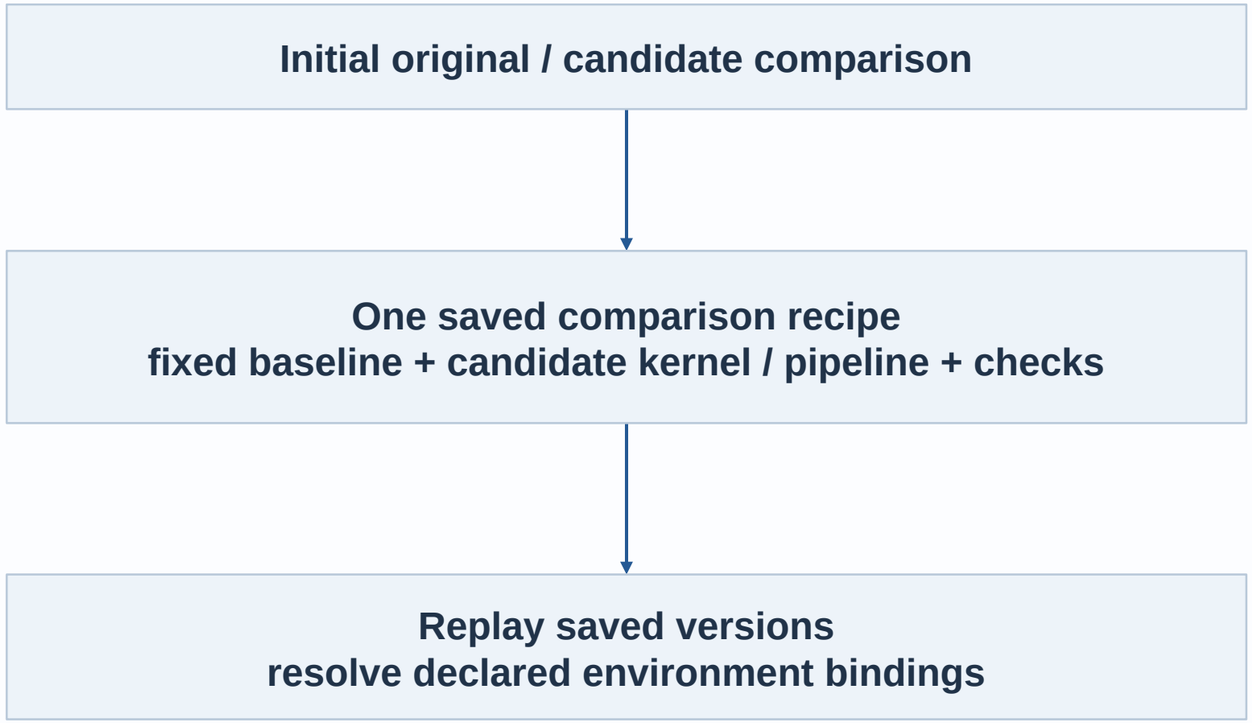


Restart → reconcile in-flight work → reuse completed stages → restore or reconstruct agent context → continue with recorded budget.

Preserve each submitted candidate and the best eligible finding independently of the current working file.

The finding is a replayable experiment

```
finding-ID/
  report.md
  manifest.json
  trial-reference.json
  original/
  candidate/
  changes.diff
  evidence/
  reproduce/
  replays/
```



Proposed interface

```
framework replay ./finding-ID
```

Each replay stores fresh check results, metric samples and resource usage linked to the original finding.

Continue the same experiment in another environment



Continuation step	Required behavior
Resolve environment	Map declared compiler / runtime / data / hardware requirements to available resources.
Restore progress	Keep IDs and budgets. Reconcile in-flight jobs, then resume compatible pending work.
Restore the agent	Resume a supported session or reconstruct context from saved messages, results and reports.
Reuse / append	Reuse compatible work only. New measurement conditions need a fresh paired comparison. Append environment-specific evidence.

```
Proposed interface: framework resume ./experiment --environment npu-lab.yaml
```

Constraints resolve into an explicit experiment profile

Constraint	Resolved setting	Failure or boundary to expose
Editing scope	Permitted IR range and pass-pipeline edits. Baseline remains fixed.	Reject edits or pass sequences outside the configured scope.
Semantics	Types, shapes, layouts, effects, aliasing, FP behavior and tolerances.	Additional agent assumptions need evidence.
Compilation	Baseline / candidate pipelines, compiler build, checkpoints and hard stop.	Unsupported entry, invalid pass prerequisites or incompatible continuation.
Evidence	Required checks; metric stage, units, target and improvement threshold.	Unsupported coverage and inconclusive outcomes.
Resources	Candidates, shared request limits, model calls/tokens and host/NPU jobs.	Stop scheduling beyond the remaining budget.

MVP scope and implementation stages

One supplied-input mode, a bounded supported pipeline space, one checking policy, one evaluator and portable experiment records.

Stage	Deliverable
Month 1	Establish pipeline entry / pass controls and checks. Trial IR / ordering alternatives on real cases.
Months 2–3	Implement variant / trial records, dialect pack, scoped edits, runners, reports and durable state.
Month 4 · MVP	Complete budgets, cross-environment continuation of pending work, finding export and replay.
Month 5	Evaluate held-out cases with agreed usefulness criteria, repeats and resource budgets.
Month 6	Compare one pass-order, context, tool, discovery or feedback refinement.

Resources — one developer, part-time compiler expertise, model/host capacity and NPU access when the selected route requires it.

Primary risks concern finding yield and integration effort

Too few useful findings

Rewrites may be invalid, trivial, narrowly applicable or already handled by a later compiler stage.

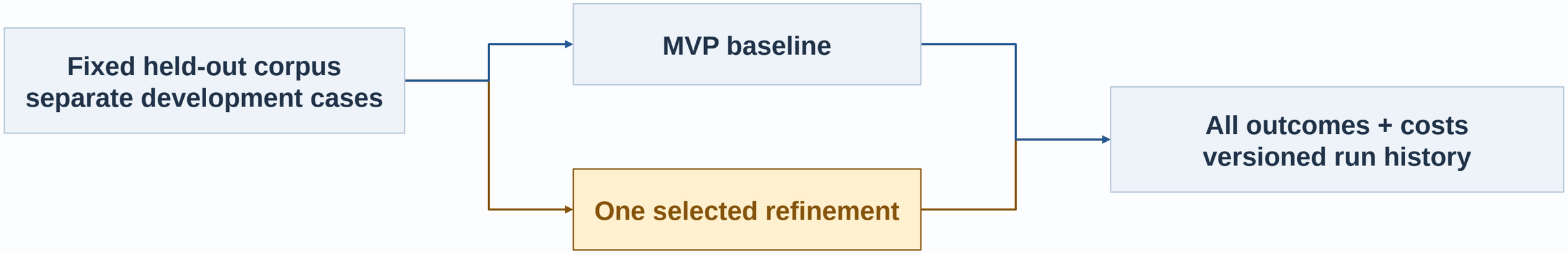
Integration costs dominate

Saved IR may require state restoration, custom harnesses, checking coverage or scarce device access.

Additional risk	Failure to watch for
Expert effort	Preparation, reviewing assumptions and maintaining the harness consume the time saved.
Metric relevance	A lower score fails to improve relevant downstream code or workloads.
Correctness coverage	Selected checks miss behavior changed by the rewrite.

Useful findings are the primary outcome. Effort saving and evidence limits are measured alongside finding quality.

Evaluation protocol and controlled refinement studies



Primary evidence	Additional outcomes
Engineer-reviewed useful findings	Rejected, duplicate, unsupported and inconclusive cases.
Replay and environment continuation	Resume pending trials elsewhere, retain provenance and rerun incompatible measurements.
At least one real case for MVP	Preparation/review effort, model use and host/NPU usage.

Before the study, agree case selection, usefulness criteria, search budgets and repeats. Any effort-saving claim identifies its reference workflow.